

Number: P3533R2
Date: 2025-02-17
Audience: CWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Changes](#)
- [Motivation](#)
 - [Example of allowed code](#)
 - [Hierarchy](#)
 - [Clang's representation](#)
- [Implementation experience](#)
- [Library impact](#)
- [Proposed changes to wording](#)
 - [Based on current draft](#)
 - [Feature test macro](#)

constexpr virtual inheritance

This paper proposes allowing usage of virtual inheritance. This will allow in future `constexpr`-ification of `std::ios_base`, and streams, removing final limitation for making most of stream formatting `constexpr`.

This will also remove definition of *constexpr-suitable* as it will be meaningless and clean all references across draft of the standard (if this paper land after [P3367: constexpr.coroutines](#)).

Changes

- [R1](#) → [R2](#): Removed wording based on [P3367: constexpr.coroutines](#) as this paper targets C++26, and `coroutines` C++29.
- [R0](#) → [R1](#): Added [alternative wording](#) in case it will land before [P3367: constexpr.coroutines](#).

Motivation

Last language syntax thing which is disallowed in [\[dcl.constexpr\]](#) which blocks us from making `std::stringstream` constant evaluable, which blocks us from making exception types for `<chrono>` (`chrono::nonexistent_local_time` and `chrono::ambiguous_local_time`). Stream formatting `<chrono>`. Using `basic_istream_view` in range code for compile-time parsing.

When we remove this limitations, language will only have limit on evaluation properties of code in [\[expr.const\]](#), and not syntactical (with exception of `reinterpret_cast`, but this is also defined as not constant evaluable.)

Then keyword `constexpr` can be seen only as an opt-in into constant evaluable and can be *later* removed and replaced with an opt-out attribute (this is not proposed here.)